

1. Variable

Variable name

Variable names include letters, numbers, and symbols. The first character must be a letter or `_`, must not contain duplicate keywords and must not contain spaces.

Variable declaration

Syntax:

```
kieu_du_lieu ten_bien1, ten_bien2, ...;
```

Variables of the same type are separated by a hyphen, and ends with `;`.

Declaring and assigning values

You can declare it first and then assign it later:

```
int a, b;
```

```
a = 1;
```

```
b = 2;
```

Or declare and assign at the same time:

```
int a = 1, b = 2;
```

2. Commands and command blocks

Command

A statement is a command that performs a task, for example:

```
x = x + 2;
```

```
printf("Hello");
```

Command block

It is a collection of commands placed within `{ }`, often written indented for readability:

```
{  
  
    a = 5;  
  
    b = 6;  
  
    printf("%d", a + b);  
  
}
```

Remember not to forget the opening and closing marks. { }.

3. Libraries in C

Concept

A library is a collection of functions, constants, and macros that are available for use in a program.

Library declaration

```
#include <ten_words.h>
```

or

```
#include "ten_words_come.h"
```

Used to insert libraries into a program.

Some popular libraries

<stdio.h> → input/output (printf, scanf)

<math.h> → mathematics (sqrt, pow, sin, ...)

<stdlib.h> → memory allocation, system processing

<string.h> → string processing

<conio.h> → Console input/output (DOS)

<graphics.h> → graphics

<stdint.h> → integer type

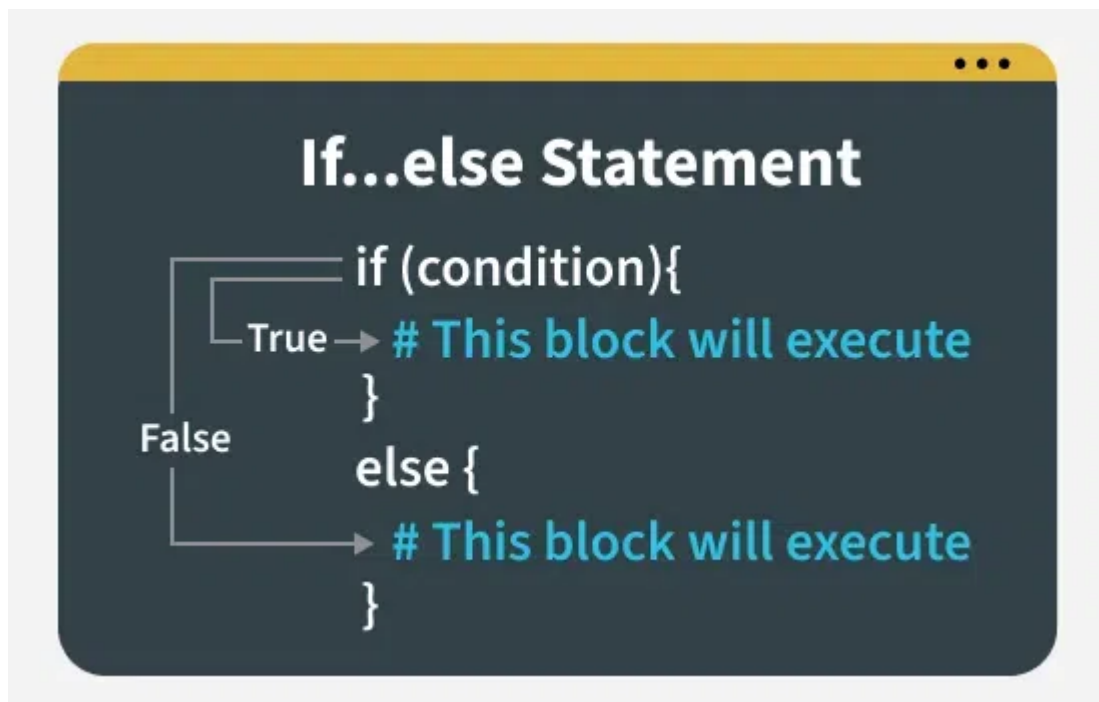
4. Conditional statements

Used to branch the program based on whether a condition is true or false.

if

```
if (dieu_kien) {  
    // execute if true  
}
```

if...else



```
if (dieu_kien) {  
    // correct  
} else {  
    // then  
}
```

if...else if...else

```

if (dieu_kien1) {
// Case 1
} else if (dieu_kien2) {
// Case 2
} else {
    // remaining
}

```

switch...case

Use when there are multiple options based on a specific value.

```

switch (good) {
    case 1:
        // handle
        break;
    case 2:
        // handle
        break;
    default:
        // default
}

```

5. Loop

Used to repeat a block of code multiple times.

for

Use this when the number of repetitions is known in advance.

```

for (int i = 0; i < 5; i++) {
    printf("%d\n", i);
}

```

while

Repeat as long as the condition remains true.

```
int i = 0;
while (i < 5) {
    printf("%d\n", i);
    i++;
}
```

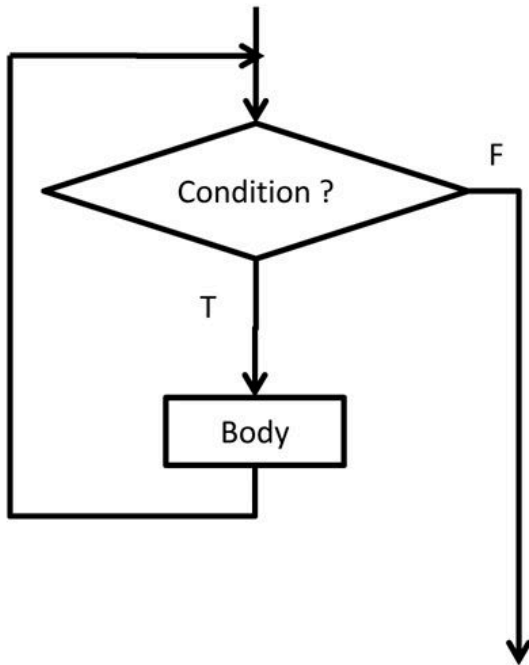
do...while

Always run it at least once before checking the conditions.

```
int i = 0;
do {
    printf("%d\n", i);
    i++;
} while (i < 5);
```

While versus Do-While Loops

while(condition)
body;



do {
body;
} while(condition);

